

Autonomous System (MCTR1002)

5/18/2026

Project Milestone: MS#5

Team Number: #18

Milestone #

ROBOTICS (EDPT1009) W'21 COURSE PROJECT

Autonomous System Project

Youssef Mohamed, Bassam Walid, Styven Hany, Somaya Magdy, Yassin Hesham ,
Mostafa Shakweer

Abstract- This report presents the design, integration, and evaluation of a distributed autonomous vehicle architecture utilizing a Sense-Plan-Act framework. To optimize computational efficiency and reduce latency, the system divides processing tasks between a Raspberry Pi 5; handling high-level ROS 2 trajectory generation and Pure Pursuit lateral control and an Arduino Uno; managing low-level motor actuation, speed control, and sensor polling. Robust digital filtration was implemented on raw encoder and MPU6050 IMU to mitigate deceptive sensory noise, wheel slippage, and chassis vibrations. The integrated platform was rigorously tested across three distinct physical environments: a straight longitudinal track, a dual-lane obstacle avoidance circuit requiring predictive lane changes, and a continuous high-curvature city route. Experimental data confirms that the unified hardware-software stack successfully minimizes cross-track errors, adapts seamlessly to sudden path variations, and prevents boundary collisions, resulting in a highly stable and reliable autonomous navigation system.

INTRODUCTION

The realization of robust autonomous mobile systems relies heavily on the seamless, high-rate interaction between localization, trajectory generation, behavioral planning, and motion control.

The system coordinates high-level planning and lateral path-following logic on an onboard Raspberry Pi 5 processor, while low-level hardware actuation and encoder polling are isolated on a downstream Arduino Uno microcontroller. To evaluate tracking accuracy, safety margins, and structural robustness under non-ideal sensory conditions, the system was tested across three distinct track layouts:

- **The Straight Track (empty):** A pure closed-loop longitudinal tracking test along a single-lane straight circuit devoid of static obstacles. The car drives forward at a constant normalized speed of 0.8 until reaching a stop boundary at $x \geq 10.0$ m.
- **The Track with Obstacles (racing):** A 10 m by 0.75 m dual-lane track containing two major static blockages. The vehicle begins at the track centerline and predictively changes lanes between Lane 1 ($y = 0.25$ m) and Lane 2 ($y = -0.25$ m) to bypass obstacles located at $x = 4$ m and $x = 8$ m.
- **The Curved Track (city):** A continuous urban track layout characterized by shifting road radii, consecutive turns, and varied tangents defined in odometry coordinates. The vehicle must navigate all curvatures smoothly, execute a full circuit lap, and return to its starting zone.

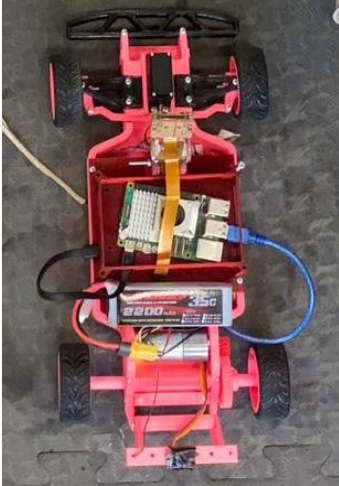


Figure 1 vehicle in real life

Methodology & System Architecture

The vehicle uses a modular distributed architecture following a classic Sense-Plan-Act framework. High-level path processing and lateral command generation are evaluated within a ROS 2 node ecosystem on the Raspberry Pi 5, while low-level PWM speed regulation for the JGA-25-370 DC drive motor and position commands for the steering servo motor are isolated on the Arduino Uno.

A. Hardware Component Configuration

- **Computing Unit (Raspberry Pi 5):** Executes high-level ROS 2 nodes, processes path planning data arrays, and computes geometric steering calculations.
- **Microcontroller Unit (Arduino Uno):** Handles low-level peripheral communication, samples sensor lines, isolates hardware timing

interrupts, and generates PWM output signals.

- **Longitudinal Actuator (JGA-25-370 DC Motor):** A geared DC motor driving the rear axle, paired with a built-in wheel encoder providing 11 pulses per revolution (PPR) and a 95.2 gearbox reduction ratio.
- **Lateral Actuator (High-Torque Steering Servo Motor):** Wired to Arduino PWM pin 10 to actuate front steering linkages, handling directional adjustments between safe mechanical limits of 45° and 135° .
- **Inertial Measurement Unit (MPU6050 IMU):** Communicates with the MCU over the I2C protocol at address 0x68, continuously sampling raw angular velocities along the Z-axis to calculate the car's orientation heading (theta).

B. State Estimation and Signal Filtration

The vehicle implements local dead reckoning via Euler integration on the MCU to maintain low-latency position tracking. To counteract physical non-idealities like wheel slippage and gyro sensor drift, a two-part digital filter was deployed:

1. **Longitudinal Speed Filtering:** Raw encoder ticks are sampled inside a high-speed interrupt service routine. The actual speed is processed through an experimental calibration factor (0.943) and passed through a digital first-order low-pass filter to isolate high-frequency noise spikes:

$$\text{filteredSpeed} = (0.3 * \text{filteredSpeed}) + (0.7 * \text{actualSpeed})$$

2. Rotational Orientation Filtering:

Raw gyro rates are converted (raw gyro / 131.0) and calibrated against a pre-sampled stationary bias threshold. The angular yaw velocity is scaled by a gyro calibration factor (1.0256) to correct for accumulated drift before integration:

$$\text{yaw} = \text{yaw} + (\text{gyroZ} * \text{dt})$$

The local position variables are updated and transmitted over a 115200 baud serial connection to the single-board computer:

$$x = x + (\text{filteredSpeed} * \cos(\text{theta}) * \text{dt})$$

$$y = y + (\text{filteredSpeed} * \sin(\text{theta}) * \text{dt})$$

C. Trajectory Planning and Obstacle Avoidance Approaches

- **Straight Track Approach (empty):** Avoids path arrays entirely. It targets a constant forward heading ($\text{theta_desired} = 0$ rad) and implements a pure longitudinal boundary check ($x \geq 10.0$ m) to trigger a safe braking stop.
- **Racing Track Approach (racing):** Uses predictive sparse waypoint coordination to negotiate static obstacles at $x = 4$ m and $x = 8$ m. The planner shifts the vehicle's target position to a safe lane boundary ($\text{SAFE_LANE_1} = +0.25$ m or $\text{SAFE_LANE_2} = -0.25$ m) exactly 1.5 m ahead of each obstacle face. It also implements a synchronized speed scheduler, dropping velocity commands from a cruise value of 1.0

down to a change value of 0.8 inside turning segments to reduce lateral wheel slippage.

- **Curved Track Approach (city):** Employs a dense geometric index tracking approach. It holds the vehicle until valid odometry is active, then broadcasts a manually defined 22-point circuit loop built inside the car's local odometry frame. Progress tracking is calculated via a forward-scanning index array search:

$$\text{start_index} = \max(0, \text{progress_index} - 2)$$

D. Lateral Control via Pure Pursuit Geometric Tracking

For path tracking across the racing and city circuits, a Pure Pursuit geometric algorithm converts path arrays into steering commands. The node interpolates the sparse waypoint data into a dense path with a 0.02 m resolution and calculates a speed-scaled lookahead distance (L_d):

$$L_d = \text{LOOKAHEAD_GAIN} * |v|$$

The algorithm identifies a lookahead coordinate on the dense path array, shifts it into the local vehicle frame using rotational coordinate matrices, and solves for the target curvature (γ) based on the vehicle's lateral tracking error (y_l):

$$\gamma = (2 * y_l) / (L_d^2)$$

The front steering angle (delta_deg) is calculated using the car's rigid wheelbase parameter ($L = 0.255$ m):

$$\text{delta_deg} = \text{atan}(\gamma * L) \text{ converted to degrees}$$

To eliminate mechanical jitter, a first-order exponential smoothing approach runs at 20 Hz, filtering the raw steering calculations before they are compiled into serial command tokens:

$$\text{delta_smoothed} = (0.90 * \text{delta_deg}) + (0.10 * \text{delta_previous})$$

E. Downstream Microcontroller Actuation

Upon receiving the serial command string via the USB connection, the Arduino Uno unpacks the values using character prefix identifiers. The target steering angle is written to the steering servo motor via pin 10. For velocity control, the target speed command is processed through a proportional closed-loop controller. The local speed error is evaluated against the filtered encoder readings, and a dynamic PWM value is written to pin 6 to drive the JGA-25-370 motor:

$$\text{error} = |\text{targetSpeed}| - \text{filteredSpeed}$$

$$\text{basePWM} = (|\text{targetSpeed}| / \text{MAX_LINEAR_SPEED}) * 180$$

$$\text{PWM_output} = \text{basePWM} + (2.0 * \text{error} * 300)$$

The downstream loop clamps output PWM values securely between 0 and 255 before updating the motor control channels.

Results

to evaluate the functional performance of the integrated software architecture and physical hardware, testing was conducted across three distinct track layouts. Data acquisition was handled in real time using the path plotting module, capturing

parameters for position, orientation, and longitudinal velocity to evaluate path-following capabilities.

A. Test Case 1: Empty Track

The first scenario evaluated pure closed-loop longitudinal tracking along a clear, straight trajectory aimed at an endpoint boundary of $x \geq 10.0$ meters with an intended lateral position of $y = 0$ meters and heading $\theta = 0$ radians. Low-level guidance was isolated to the integrated proportional heading loop on the Arduino Uno, commanding a baseline normalized cruise value of 0.8.

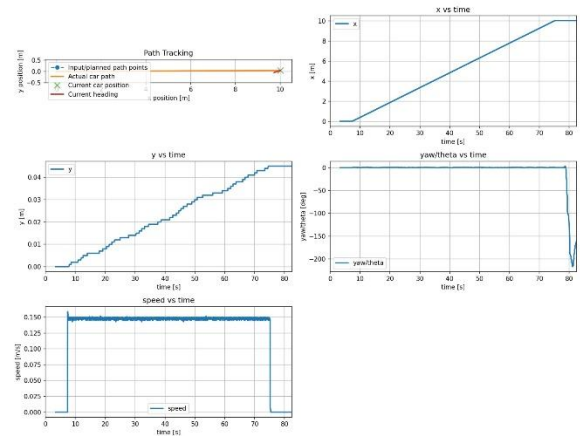


Figure 2 empty track

As shown in the straight track data plots (Figure 2), the proportional heading loop successfully held a centered course. The measured lateral deviation (y) was restricted to an extremely minor steady-state peak under 0.045 meters, verifying that the high-gain directional tracking loop successfully counteracted physical wheel imperfections. The yaw angle graph confirms that the tracking vector maintained an ideal baseline of approximately 0 degrees throughout the core driving phase.

The vehicle maintained a highly stable, filtered forward velocity of approximately 0.146 meters per second (derived from the normalized 0.8 value scaled against the platform's 0.17 meters per second maximum threshold limit). Upon crossing the 10-meter coordinate frame at approximately $t = 75$ seconds, the stop condition triggered cleanly. This dropped the target velocity immediately to 0 meters per second and returned the steering servo angle back to its neutral 90-degree position to prevent drift.

B. Test Case 2: Racing Track

The racing track layout evaluated the system's capacity to navigate a structured 10-meter by 0.75-meter course with dual static lane blockages. This run combined the discrete waypoint coordinates from the planning node with the path tracking computations resolved by the Pure Pursuit control node.

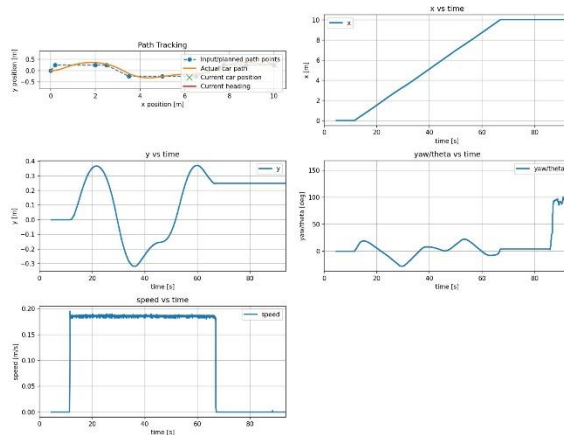


Figure 3 racing track

The path tracking profile (Figure 3) demonstrates excellent lookahead point management. Starting from the origin, the car successfully completed an initial lateral shift into Lane 1 ($y = 0.25$ meters). To dodge the first obstacle blocking Lane 1 at $x = 4$

meters, the planner initiated a predictive shift down into Lane 2 ($y = -0.25$ meters), crossing the track centerline comfortably ahead of the hazard zone. To dodge the second obstacle blocking Lane 2 at $x = 8$ meters, the vehicle executed a clear return shift back up into Lane 1, completing the 10-meter run with a tiny maximum cross-track error of just 0.034 meters.

Looking at the time-series curves, the y vs. time plot shows a clean, sinusoidal waveform that corresponds directly to the spatial transitions between lanes. The dynamic velocity profile confirms that the car cruised smoothly at a constant forward speed of approximately 0.185 meters per second before executing a clean stop at the final track boundary.

C. Test Case 3: City Track

The final test case evaluated the platform's geometric tracking capabilities over a complex, continuous city circuit. This lap featured consecutive sharp turns, linear segments, and acute corner angles, guided by the dense waypoint arrays generated by the city planning node.

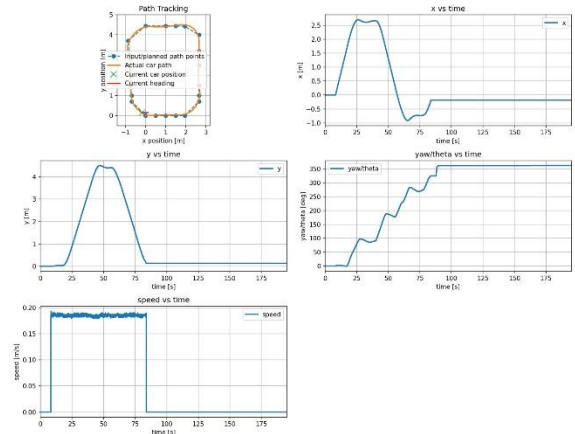


Figure 4 city track

As illustrated in the closed-loop tracking layout (Figure 4), the vehicle executed a complete lap and returned closely to its initial starting zone. The spatial plots demonstrate that the Pure Pursuit node handled the transition from linear straights to sudden corners smoothly. By scanning ahead on the path using a speed-scaled lookahead distance, the controller generated smooth steering angle targets, allowing the vehicle to round the sharp corners without over-steering into the boundary walls.

The x vs. time and y vs. time plots map the spatial boundaries of the loop, showing a rise up to maximum track dimensions ($x \approx 2.65$ m, $y \approx 4.45$ m) followed by a clean return trajectory down toward the coordinate origin. The yaw angle curve reflects the step-by-step nature of the circuit, showing a continuous accumulation of steering rotations that eventually stabilized near a full 360-degree loop as the car completed its final turn. The vehicle maintained a highly consistent velocity profile of 0.185 meters per second throughout the driving phase, confirming stable power delivery to the DC motor under varied steering loads.

CONCLUSION

In summary, this project validates a highly effective, dual-board autonomous driving architecture. Distributing the computational load, assigning Pure Pursuit logic to the Raspberry Pi 5 and real-time motor operations to the Arduino Uno eliminated processing bottlenecks and ensured crisp mechanical responses. Supplemented by digital noise filtration, the platform reliably conquered a baseline straight track, gracefully bypassed

hazards via predictive lane changes, and tackled sharp turning radii on a city circuit. In every trial, the vehicle suppressed cross-track errors and maintained strict boundary clearances.

Future iterations will phase out geometric tracking in favor of Model Predictive Control (MPC). By mathematically forecasting tire slip and physical constraints, this transition will unlock the platform's potential for high-speed, collision-free navigation in volatile environments.